

Frontend Assignment: Product Admin Dashboard

Next.js • React • Tailwind CSS • Axios | Time: 2 days

Build a small admin dashboard where a user can log in and manage products. Use the free DummyJSON API (<https://dummyjson.com>). All API calls must use Axios.

What to build

- **Login page:** log in with username *emilys* and password *emilyspass* (POST /auth/login). Show errors for wrong details. Only logged-in users can open the product pages. Add a logout button.
- **Product list:** show image, title, category, price, rating and stock. Use a table on desktop and cards on mobile.
- **Pagination:** load data page by page from the API using limit and skip. Show page numbers, Previous/Next buttons, a page size option (10, 20, 50) and text like “Showing 21–40 of 194”.
- **Search:** search products with /products/search?q=. Wait until the user stops typing before calling the API. Go back to page 1 when the search changes.
- **Filter and sort:** filter by category (/products/categories) and sort by price, rating or title.
- **Product details:** a page at /products/[id] with images, description, price and reviews. Show a “not found” page for a wrong id.
- **Add, edit and delete:** a form with validation to add or edit a product, and a confirm popup before deleting.
- **Loading, empty and error states:** show a loader while loading, a message when nothing is found, and a Retry button when something fails.

Rules

- Create one shared Axios setup file. It should add the login token to every request and handle errors in one place.
- Keep the page, search, filter and sort values in the URL, so refreshing the page or sharing the link shows the same result.
- Do not use React Query, SWR or ready-made table/pagination libraries. Write the logic yourself.
- Keep components small and put API calls in separate files, not inside the UI code.

Things to handle carefully

- If the user types fast, old search results must never replace new ones. (Test by adding &delay=2000 to the API URL.)
- The API cannot search and filter by category at the same time. Decide what your app does and explain why.
- Add, edit and delete are not really saved by the API. Show the change in the app anyway and explain your approach.
- Wrong URL values like ?page=abc or ?page=999 must not break the page.
- Clicking Save or Login many times quickly must not send many requests.

AI tools

You can use AI tools. But you must understand and explain every line of your code. In the next round, you will walk us through your code and make a small change to it live.

What to submit

- A public GitHub repo with regular commits (not one big commit).
- A live link (Vercel or Netlify).
- A README with setup steps and a list of what you finished.
- A short note explaining your choices, one problem you faced and how you fixed it, and where AI helped you.